CS438: Secure Distributed Computing

Yonathan Lanzmann
yonathan.lanzmann@epfl.ch

Gaël Conde Losada
gael.condelosada@epfl.ch

Alexandre Mourot
alexandre.mourot@epfl.ch

I. Executive Summary

Federated learning enables collaborative training of machine learning models without sharing raw data, addressing privacy concerns and regulatory requirements. However, it faces challenges in securing data during computation, especially against adversarial or untrusted nodes. Homomorphic encryption (HE) provides a robust solution by enabling computations on encrypted data, ensuring confidentiality throughout the training process.

This project implements a secure distributed computing framework for federated learning using homomorphic encryption. By leveraging HE, the system ensures data remains encrypted, preserving privacy and preventing breaches. The framework's design separates protocol development, model training, and encrypted aggregation.

Key components ensure secure and efficient federated learning:

- **Decentralized Network Protocols**: Built from HW0 and HW1, establishing communication channels between nodes using routing and rumors mechanisms.
- **Encryption Module**: Implements xMK-CKKS homomorphic encryption for approximate arithmetic on encrypted data, ensuring computations occur without decryption while encoding and decoding data into the encryption field.
- **Machine Learning Module**: Provides training and prediction functionality for models on encrypted data, supporting gradient computation and secure updates.
- **Controller Module**: Coordinates federated training, managing rounds, distributing updates, and aggregating results received from the network. Each node can act as a Worker or Controller (`TrainFederated` function in `/peer/impl/node.go`).
- **Worker Module**: Handles local training and participation in federated learning processes.

The project is implemented on top of the peerster system. The xMK-CKKS scheme and matrix operations for model training utilize the [1] and [2] libraries, respectively. Nodes act as both federators (controllers) and trainers (workers), ensuring secure local computations and communication. The implementation follows the methodology in [3], adhering to best practices for federated learning with homomorphic encryption and a modular, scalable design.

Testing includes functional (unit), integration, and performance evaluation. Functional tests verify computation accuracy and aggregation fidelity. Performance tests assess scalability, protocol overhead, and model accuracy under various conditions, ensuring a reliable and efficient system.

II. Related Work

We reviewed the literature on federated learning and homomorphic encryption to identify key advancements relevant to our project. The study *Federated Learning with Homomorphic Encryption* [3] provides the foundation for our project, offering the protocol for secure encrypted model updates and aggregation. *Privacy-Preserving Federated Learning Using Homomorphic Encryption With Different Encryption Keys* [**?**] explores using distinct encryption keys to enhance privacy, analyzing trade-offs in scalability and computational efficiency.

Efficient and Privacy-Preserving Federated Learning Based on Full Homomorphic Encryption [4] focuses on practical implementations of full homomorphic encryption, emphasizing efficiency and real-world applicability. *ESAFL: Efficient Secure Additively Homomorphic Encryption for Cross-Silo Federated Learning* [5] introduces a tailored encryption scheme optimized for performance in cross-silo federated learning scenarios. Lastly, *Privacy-Preserving Federated Learning with Homomorphic Encryption and Sparse Compression* [**?**] addresses cost reduction using sparse compression techniques while safeguarding privacy against model inversion and inference attacks.

III. Threat Model

Our threat model assumes that nodes in the network are "honest but curious". This means that nodes act as required by the protocol but may attempt to infer private information about other devices from the data shared during the execution of the protocol. To mitigate this, we make use of homomorphic encryption scheme. Homomorphic encryption possesses a highly useful property that ensures certain operations (in our case addition) performed on ciphertexts translate in the corresponding plaintexts. Formally, for plaintexts $m_1, m_2 \in \mathcal{P}$, ciphertexts $\mathcal{E}(m_1), \mathcal{E}(m_2) \in \mathcal{C}$, and operations $\circ$ (plaintext) and $\star$ (ciphertext), it satisfies:

$$\mathcal{D}(\mathcal{E}(m_1) \star \mathcal{E}(m_2)) = m_1 \circ m_2$$

This enables the controller to operate on ciphertexts without compromising the integrity of the plaintexts obtained upon

decryption. Since the controller only interacts with encrypted elements until they are aggregated, it becomes difficult for the controller to infer any information about the individual datasets contents.

We say it is only difficult, because recent studies have highlighted potential risks of privacy leakage even when homomorphic encryption is used. For instance, [6] argues that the controller could infer private information from aggregated weights, and [7] demonstrates how model updates exchanged during federated learning may indirectly leak sensitive data. These findings underscore the need for advanced privacy-preserving mechanisms to complement existing encryption techniques. However, because these findings go beyond the scope of our project, we did not implement their proposed solutions.

Furthermore, we emphasize that our primary concern is ensuring the privacy of data content, not its size. Our focus is on implementing the theoretical principles of homomorphic encryption within the context of federated machine learning. Consequently, we view a "curious" node as one that aims solely to uncover the individual data contents.

IV. Design

In this section, we describe the design of our system, outlining its high-level structure and detailing the components of our work. Figure 1 illustrates the high-level architecture of our system.

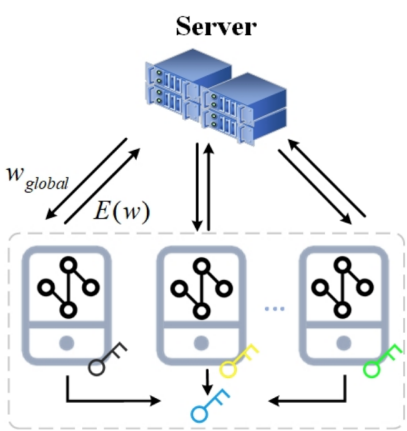

Fig. 1: High-Level Architecture of the System [3]

A. Decentralized Network Protocols

The distributed network protocols module, implemented in HW0 and HW1, provides the foundational communication infrastructure for the system. Since this component was already built in earlier assignments, we do not describe it here. However, it is important to note that we reuse only the underlying structure developed in HW0 and HW1, excluding HW2 and HW3, as they are irrelevant to our project.

B. Encryption Module

The encryption module is responsible for managing cryptographic operations, specifically using homomorphic encryption. This module is implemented in the file `/peer/impl/securecomputing/cryptography.go` and includes the `Encryptor` structure. The `Encryptor` provides functions for encryption, key aggregation, decryption of shares, and others. It also includes encoding and decoding functionalities to bridge the mismatch between the data field and the cryptographic field.

Since data dimensions may vary across training instances, the underlying cryptographic field must also be unique for each instance. Therefore, the `Encryptor` object is designed to be conceptually tied to a specific training instance. This design choice simplifies handling and ensures scalability when multiple training instances are involved. We call a training instance the entire process for training the model. A more precise definition is given in Section V.

C. Machine Learning Module

The machine learning module is centered around the `Model` interface, defined in `/types/model.go`. This interface abstracts the concept of a machine learning model, facilitating modularity and scalability. The `Model` interface specifies several methods, including `Train` and `Predict`, which are crucial for federated learning workflows.

The abstraction provided by the `Model` interface allows for seamless integration of different machine learning models, making it easier to extend the system for various use cases. This modularity ensures that new models can be incorporated with minimal changes to the existing codebase.

D. Controller Module

The controller training function, implemented in the `TrainFederated` function of `/peer/impl/node.go`, orchestrates the entire federated learning process. In our design, we consider a controller node a node that initiates and manages federated training, while the remaining nodes in the network act as workers. For simplicity, we assume that the controller and workers are disjoint entities, though in practice a node can fulfill both roles, as we cover it in Section V.

In our design, nodes do not share raw data to each other. Unlike other implementations in the literature, our controller does not send any training data but instead specifies the data type to be used and the initial weights. Nodes perform local training on their own dataset according to the data type and initial weights received from the controller, and send back the encryption of resulting weights from training. The controller aggregates the cipher texts it receives, and uses the decryption shares of the participating nodes to decrypt the aggregation. The plain text decrypted corresponds to the sum of the weights computed by the participating nodes. Finally, the controller retrieves the model weights by computing the average of the aggregated weights.

This is done for a given number of iterations, to improve the machine learning model's performance.

Furthermore, we impose a constraint that a node can control only one training instance at a time. In other words, a node cannot act as the controller for two or more different models simultaneously. It must wait for the current training process to complete before initiating a new one.

E. Worker Module

This module is quite simple. It describes how nodes behave when they receive messages regarding federated training. Naturally, when nodes are training locally for a controller, they do so asynchronously (in a new go routine) so that they don't block the packets handler. The explanation in depth is given in the following section.

V. Implementation

In this section, we provide a detailed explanation of the implementation. We discuss the technical and design decisions made throughout the process and demonstrate the implementation of each component. Again, we do not provide details about the underlying Peerster system.

A. Encryption Module

As discussed earlier, the encryption module provides the Encryptor object, which manages all cryptographic operations in the project. It leverages the Lattigo library [1] for its ability to handle polynomial rings, the foundational structure for the homomorphic cryptographic operations used. For the context of our project, a constant and sufficiently large ring size was chosen.

Since most of the functions are direct implementations from [3], we do not describe them in detail here. Instead, we focus on our contributions and the rationale behind our parameter choices.

The encoding functionality provided by the Encryptor is central, as it determines the size of the polynomial on which the cryptographic computations are performed. While the encoding process may appear straightforward, it poses certain challenges, which are discussed further in Section VIII. The encoding process creates a polynomial whose coefficients correspond to the elements of the matrix to be encoded (remember that the elements to be encrypted, and therefore encoded, are the weights). This requires a shift operation to convert the float values (weights) into integer values (polynomial coefficients) with a specific precision, defined as a constant. Decoding reverses this process to retrieve the original values.

Given this encoding scheme, it is evident that when initializing an Encryptor object, which sets up a polynomial ring, the size of the polynomial depends directly on the dimensions of the data. Specifically, it corresponds to mn, where m and n are the rows and columns of the weights, respectively.

Once an Encryptor object is initialized, it allows for the generation of key pairs, which are essential for encryption and decryption of shares. Notably, the Encryptor data structure also includes a function to update the public key. While this may seem unusual, it is necessary for updating the aggregated key, a process explained in more detail later.

As described in [3], the encryption relies on error samples, which can be interpreted as noise, to ensure privacy. Since the paper does not specify a precise distribution for these error samples, we opted for a Gaussian distribution with a standard deviation of 2.5. This choice is made to balance the trade-off between privacy and decryption accuracy. Larger errors provide greater privacy but reduce decryption accuracy, while smaller errors compromise privacy for the expense of better decryption accuracy. Through experimentation, we determined that a standard deviation of 2.5 achieves the best balance for our use case. Larger standard deviations resulted in decryptions deviating significantly from expected values. Since no specific distributions were provided in the paper, we implemented our own approach for this aspect of the system.

B. Machine Learning Module

We focus solely on the very basic implementation of linear regression. For this purpose, we define a LinearModel structure in `/peer/impl/securecomputing/models`, which implements the Model interface. Upon initialization, the model generates random weights with the specified dimensions.

To facilitate extensions of the project, we implement a gradient descent algorithm that can be used by general models, given a specific loss function. In this project, since we only implement linear models, the gradient descent algorithm is applied to this basic linear model.

After `Train` function is executed, the model's weights are updated with the resulting values. Predictions on a given dataset are then made using the `Predict` function, which outputs the corresponding results.

The implementation relies on the matrix and vector operation capabilities provided by the Gonum library [2]. This library simplifies mathematical computations allow an efficient implementation of the basic operations.

C. Controller Module

The global workflow is structured around what we call multiple message-response blocks, where a block consists of a controller sending a message to the network and collecting responses. Furthermore, the node initiating the protocol uses the `Controller` data structure defined at `/peer/impl/securecomputing/controller.go`.

The workflow proceeds as follows : The controller initiates federated learning for a specific data type. Recall that a controller can manage only one training instance at a time. We call a training instance the entire lifecycle of the `TrainFederated` function. This means that a node can not execute the `TrainFederated` function twice or more in parallel. 1- The controller broadcasts the network for nodes willing to participate in the training instance. Nodes respond if they choose to participate. The controller collects the responses, determines the set of participating nodes and computes the aggregated public key. (note that step 1 is an example of message-responses block, as defined above). 2- The controller sends a message to the accepted nodes, instructing

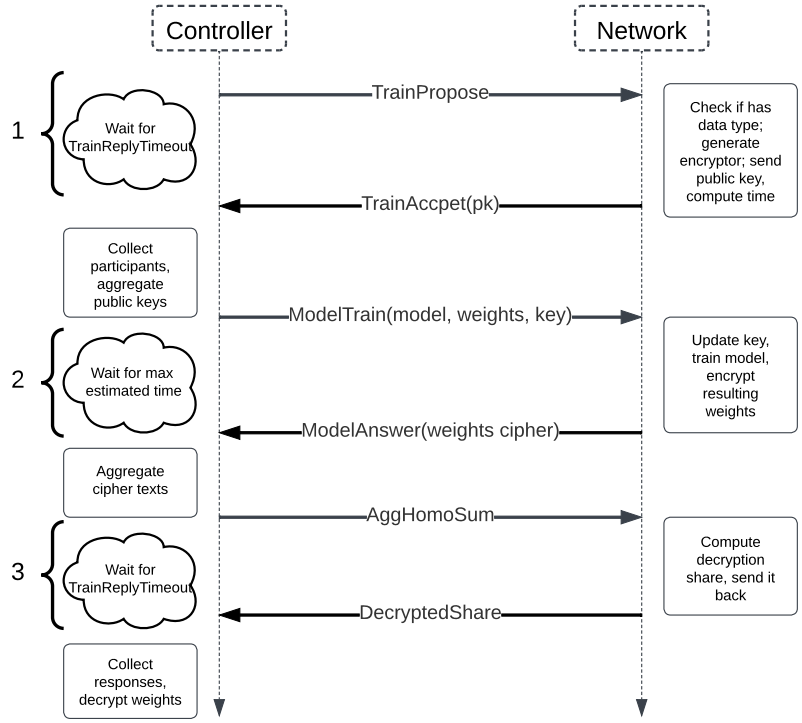

Fig. 2: Complete workflow of the protocol, if all participating nodes answer.

them to start local training, and sharing them the aggregated public key. Once completed, the nodes send their encrypted weights back to the controller. The controller aggregates the encrypted weights and decrypts the result after a third (3 -) round of message-responses block used for aggregating the decryption shares. Diagram 2 represents this workflow.

This concludes the federated training workflow, which is a direct implementation of the chosen paper [3]. However, some important points remain unaddressed.

First, handling nodes going offline or failing to respond presents a challenge. The cryptographic framework requires all nodes that participate initially to also respond in the final steps. If a node fails to send its encrypted weights or decryption share during the last two message-response blocks, the controller must restart the entire training process.

Second, determining how long the controller should wait for nodes to respond (when they finish training) is another issue. Without knowledge of the dataset sizes or computational power of the nodes, defining an appropriate waiting period is challenging. To mitigate this, each node estimates the maximum time it needs for local computation, and the controller waits for the longest estimate. While this approach prevents indefinite delays, it leaks some information about dataset sizes, as differences in estimated times could reveal relative sizes. However, as outlined in the threat model, this type of information leakage is not considered within the scope of a ”curious” node.

To address delays, a backoff strategy is implemented. If responses are not received within the maximum estimated time, the controller restarts the process with an increased backoff factor, allowing additional time for nodes to respond during subsequent attempts.

D. Worker Module

In this section, we define the behavior of nodes when receiving messages related to federated training, as illustrated in Figure 2.

When a node receives a TrainPropose message, it accepts to be a worker if it possesses the required data type. If the data type matches, the node estimates its computing time, generates key pairs, and sends its public key to the controller. Recall that an `Encryptor` object is tied to the training instance. We use the `WorkManager` structure, defined in `/peer/impl/securecomputing/worker.go`, to implement that binding.

Upon receiving a ModelTrain message, the node, now functioning as a worker, trains the specified model on its local dataset. It encrypts the resulting weights using the aggregated public key received earlier, which is stored in its associated Encryptor.

When a node receives an AggregateSum message, the worker decrypts its share of the weights and sends the decrypted share back to the controller. This process ensures the integrity of the aggregated model while maintaining the privacy of individual contributions.

VI. Testing

We prioritize unit testing early in development to identify and fix bugs before they become deeply integrated in the codebase. Unit tests are often implemented shortly after their

corresponding features, or, in some cases like the cryptographic module, developed simultaneously with the feature. This approach streamlines development and helps resolve issues efficiently.

Testing primarily focuses on identifying bugs and evaluating scalability. To ensure objectivity, features and their associated tests are typically developed by different team members. This reduces the risk of bias when making tests.

Then, once the main protocol is implemented, we conducted the integration tests running the full protocol to validate the system's overall workflow. These tests uncover a few protocol-level issues missed by unit tests, all of which are resolved promptly. As the system stabilizes, additional tests are added to cover edge cases, including stress scenarios and scenarios over jammed networks. These tests help identify areas to improve performance and robustness.

Finally, we implemented performance tests to assess the system efficiency. These tests provide insights into system behavior under various conditions, with no unexpected results observed during execution.

VII. Evaluation

The evaluation focuses on three aspects: scalability, communication overhead, and stress handling, ensuring the system performs reliably under varying conditions.

Scalability is tested by increasing the number of nodes up to 1000 (not left in the tests as it is very time-consuming) to determine if the system remains robust as the network grows. This verifies that model training stays consistent and response times do not degrade significantly. We mostly use the `Unicast` function and not `Broadcast` to enhance performance. We allow ourself to do so as we are not concerned by packets loss, thanks to the model's design. Figure 3 illustrates the training time of the model as the number of nodes increases, while keeping the total dataset size constant. As expected, the training time starts to increase after few nodes in the network, primarily due to the communication overhead introduced by the federated learning protocol.

Communication overhead measures the total size of messages exchanged during federated learning. This analysis identifies the protocol's efficiency and potential communication bottlenecks.

Stress handling tests the system under extreme communication loads by simulating all nodes acting as controllers simultaneously. This evaluates the system's ability to handle high volumes of concurrent communication without instability.

The evaluation prioritizes a functional and robust architecture over precise predictive accuracy, ensuring seamless implementation with acceptable performance.

We tested simple machine learning models using a binary framework to verify functionality, focusing on correct predictions rather than optimizing for complex scenarios. We also assessed the system's ability to generalize across diverse data distributions, evaluating if the trained model could recognize distinct types of data in different topologies.

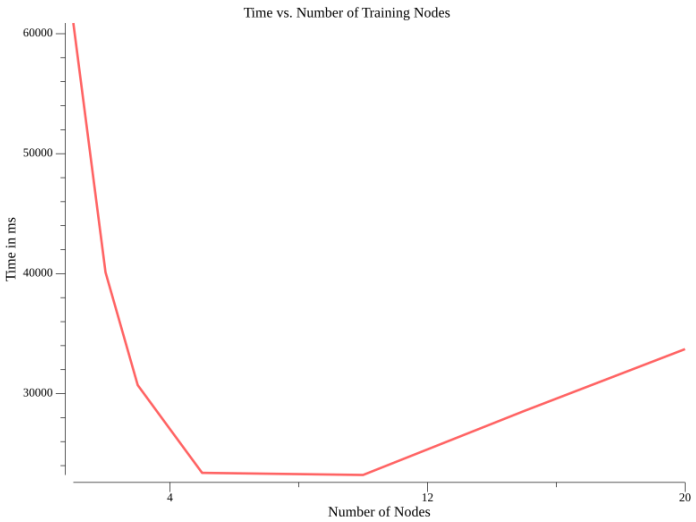

Fig. 3: System performance as a function of the number of nodes.

To test resilience, we introduced noisy data to some nodes and observed its diminishing effect as more nodes participated. This demonstrated the aggregation process's robustness and its ability to mitigate outliers.

Lastly, we evaluated the protocol's overhead by comparing federated and centralized training. These tests revealed trade-offs between security and efficiency as data and model sizes grew.

VIII. Limitations & Future work

A key limitation is the weight matrix encoding: adding noise to near-zero weights can result in negative values that, when reduced modulo the ring size, produce invalid encoded outputs, causing decoding failures. Developing a more robust encoding scheme is essential.

Future work could focus on extending the system to support more complex machine learning models, such as logistic regression or neural networks. Additionally, refining the definition of a "curious" node by incorporating dataset size considerations could strengthen privacy guarantees.

References

[1] J.-B. Orfila, T. Minier et al., "Lattigo: Go library for homomorphic encryption," https://github.com/ldsec/lattigo, 2020.

[2] T. G. Authors, "Gonum: A set of numeric libraries for go," https://gonum.org, 2019.

[3] X. Gong, X. Zhang, Z. Guo, Z. Lu, and P. Xiong, "Federated learning with homomorphic encryption," arXiv preprint arXiv:2104.06824, 2021.

[4] A. Unknown, "Efficient and privacy-preserving federated learning based on full homomorphic encryption," arXiv preprint arXiv:2403.11519, 2024.

[5] J. Wu et al., "Esafl: Efficient secure additively homomorphic encryption for cross-silo federated learning," arXiv preprint arXiv:2305.08599, 2023.

[6] A. Unknown, "Fhefl: Fully homomorphic encryption friendly privacy-preserving federated learning with byzantine users," arXiv preprint arXiv:2306.05112, 2023.

[7] ——, "Fedshe: Privacy-preserving and efficient federated learning with adaptive segmented ckks homomorphic encryption," SpringerOpen Cybersecurity, 2024.